МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №2

з курсу: „Паралельні обчислення”
тема: “ Дослідження базових примітивів синхронізації”

виконав студент 3го курсу
групи ДА-11
Сидько Нікіта Сергійович

КИЇВ – 2024

Мета роботи: Розглянути базові примітиви синхронізації та їх особливості, в залежності від обраної мови програмування. Розглянути підходи до побудови ПЗ з використанням паралелізму та ознайомитися з класичною задачею паралелізму у вигляді пулу потоків.

Завдання

1. Ознайомитися з визначенням поняття пул потоків (thread pool), використовуючи даний методичний посібник, або ж сторонні джерела. Ознайомитися з примітивами синхронізації та проблемами, пов'язаними з ними.

2. Реалізувати власний пул потоків з характеристиками, зазначеними в обраному варіанті. Обов'язкові для кожного варіанту характеристики: пул потоків повинен бути написаним коректно відносно обраної мови програмування, повинен мати можливість коректного завершення своєї роботи (моментально, з покиданням всіх активних задач, так і з завершенням активних задач), можливість тимчасової зупинки своєї роботи, працювати з використанням умовних змінних)

3. Операції ініціалізації та знищення пулу, додавання та вилучення задач в чергу повинні бути безпечними з точки зору паралелізму.

4. Створити програму, котра буде виконувати задачі за обраним варіантом, використовуючи написаний студентом пул потоків. Код відповідальний за додавання задач в пул потоків, та сам пул потоків повинні знаходитися в різних потоках виконання. Задачі в пул потоків повинні додаватися з декількох потоків.

5. Перевірити та довести коректність роботи програми з використанням системи вводу/виводу інформації в консоль (або іншого засобу – профайлера).

6. Зробити обмежене за часом тестування та розрахувати кількість створених потоків та середній час знаходження потоку в стані очікування. Для задач з необмеженою чергою – визначити середню довжину кожної черги та середній час виконання задач. Для обмежених за розміром черг – визначити максимальний та мінімальний час, поки черга була заповнена, кількість відкинутих задач.

7. В протоколі роботи описати ключові моменти реалізації пулу потоків в цілому, а також ключові моменти при імплементації конкретного варіанту завдання. Описати публічний інтерфейс пулу потоків та механізм його роботи. Описати механізм тестування розробленого рішення. Значення метрик, перечисллений в пункті 6 для різних проміжків часу, обраних в довільному порядку самим студентом.

8. Надати висновок, що повинен містити аналіз отриманих результатів.

Варіант 24

Пул потоків обслуговується 8-ма робочими потоками й має одну чергу виконання. Задачі додаються одразу в кінець черги виконання. Задача береться на виконання з буферу одразу за наявності віль ного робочого потоку. Задача займає випадковий час від 10 до 16 секунд.

Пул потоків (thread pool) - це механізм у програмуванні, який використовується для оптимізації управління потоками. У ньому створюється певна кількість потоків, які виконують задачі у пулі, замість того, щоб створювати новий потік кожного разу, коли потрібно виконати задачу.

М’ютекс (Mutex) – примітив синхронізації, головною задачею якого є бмеження одночасної кількості потоків виконання всередині процесу, що мають доступ до певної ділянки коду. М’ютекс обмежує доступ до ділянки коду лише для одного потоку виконання. Класичний інтерфейс м’ютекса складається з двох методів (або функцій, в залежності від мови): lock та unlock. Потік виконання, що викликає метод lock, стає власником м’ютекса, і допоки власник не викликає метод unlock (звільнить м’ютекс), виклики до методу lock з інших потоків змусять їх чекати виклику unlock від потоку власника. Виклик методу lock два рази підряд з одного потоку виконання в більшості випадків приводить до помилки часу виконання.

Семафор (Semaphore) – примітив синхронізації, схожий за концепцією на м’ютекс, проте обмежує доступ до секції коду для певної кількості потоків виконання. В деяких мовах програмування наявний в нативному вигляді, або представлений у вигляді платформних бібліотек. В інших – відсутній, проте може бути реалізований за допомогою наявних примітивів синхронізації та лічильника, що відслідковує кількість потоків, що викликала методи lock та unlock.

Умовна змінна (Conditional Variable) – примітив синхронізації, котрий блокує виконання потоку (або декількох потоків), допоки інший потік не виконає певну умову (condition) і не оголосить про це (повідомить про це потоки виконання, що чекають).

У даній лабораторній роботі було реалізовано пул потоків, який складався з 8 виконавчих потоків, що брали задачі з черги. Клас ThreadPool відповідає за створення і керування робочими потоками, які виконують завдання з черги.

```cpp
class ThreadPool {
private:
    std::vector<std::thread> workers;
    TaskQueue& queue;
    std::vector<double> total_execution_time;
    std::vector<int> execution_count;
public:
    ThreadPool(TaskQueue& q, int workers_count) : queue(q) {
        total_execution_time.resize(workers_count, 0.0);
        execution_count.resize(workers_count, 0);

        for (int i = 0; i < workers_count; i++) {
            workers.emplace_back([this, i]() { this->worker(i); });
        }
    }
```

Конструктор ThreadPool, ініціалізує пул з вказаною кількістю робочих потоків (workers_count), кожен з яких буде брати завдання з переданої черги (queue). Вектори для зберігання часу виконання та кількості завдань ініціалізуються для кожного потока.

```cpp
void worker(int id) {
    while (!queue.isStopped()) {
        Task task = queue.getTask();
        if (task.func) {
            std::cout << "Завдання №" << task.id << " початок виконання" << std::endl;
            auto start_time = std::chrono::high_resolution_clock::now();
            task.func();
            auto end_time = std::chrono::high_resolution_clock::now();
            std::chrono::duration<double> elapsed = end_time - start_time;
            total_execution_time[id] += elapsed.count();
            execution_count[id]++;
            std::cout << "Завдання №" << task.id << " виконано за " << elapsed.count() << " секунд" << std::endl;
        }
    }
}
```

Кожен потік виконує цикл, поки черга активна, обробляючи завдання, які він отримує. Час виконання кожного завдання фіксується.

```cpp
void stop() {
    queue.stopQueue();
    for (auto& worker : workers) {
        if (worker.joinable()) {
            worker.join();
        }
    }
}
```

Метод stop, зупиняє всі потоки після завершення роботи черги та гарантує що всі потоки завершать свою роботу перед закриттям програми.

```
void printStatistics() {
    double total_time = 0;
    int total_tasks = 0;

    for (int i = 0; i < total_execution_time.size(); i++) {
        total_time += total_execution_time[i];
        total_tasks += execution_count[i];
    }
    std::cout << "Сумма часу всіх задач: " << total_time << " секунд" << std::endl;
    std::cout << "Загальна кількість виконаних завдань: " << total_tasks << std::endl;
    if (total_tasks > 0) {
        std::cout << "Середній час виконання задачі: " << (total_time / total_tasks) << " секунд" << std::endl;
    }
}
```

Виводить статистику по часу виконання і кількості завдань.

3. Операції ініціалізації та знищення пулу, додавання та вилучення задач в чергу повинні бути безпечними з точки зору паралелізму.

Також у роботі був реалізований клас TaskQueue який відповідає за управління чергою завдань. Використання мьютексів та умовних змінних забезпечує безпечне додавання та вилучення завдань у чергу без конфліктів та гонок даних.

```
class TaskQueue {
private:
    std::queue<Task> tasks;
    std::mutex queue_mutex;
    std::condition_variable cond_var;
    bool stop = false;
    bool paused = false;
    int task_id = 0;

public:
    void addTask(const std::function<void()>& task) {
        std::unique_lock<std::mutex> lock(queue_mutex);
        if (!stop) {
            tasks.emplace(task, ++task_id);
            std::cout << "Завдання №" << task_id << " додано в чергу" << std::endl;
            cond_var.notify_one();
        }
    }
```

Метод addTask :
Використовує std::unique_lock<std::mutex> для блокування queue_mutex. Це гарантує, що тільки один потік може додавати завдання в чергу в будь-який конкретний момент часу після чого перевірка (чи не було чергу зупинено)
Якщо черга не зупинена, завдання додається до черги з використанням методу emplace.
Виклик notify_one на умовній змінній cond_var для сповіщення одного з потоків, що чекають на завдання, про наявність нового завдання.

```
Task getTask() {
    std::unique_lock<std::mutex> lock(queue_mutex);
    cond_var.wait(lock, [this]() { return stop || !tasks.empty() || paused; });
    if ((stop && tasks.empty()) || paused) {
        return Task(nullptr, 0);
    }
    Task task = tasks.front();
    tasks.pop();
    return task;
}
```

Метод getTask:
Блокування м'ютексу (std::unique_lock<std::mutex>) для безпечного доступу до черги. cond_var.wait блокує потік, поки не буде виконана умова для продовження (черга зупинена, є доступні завдання, або черга на паузі). Якщо черга зупинена і пуста, або на паузі, повертає пусте завдання.
Повернення завдання - витягує завдання з переду черги та видаляє його з черги.

```
void pause() {
    std::unique_lock<std::mutex> lock(queue_mutex);
    paused = true;
    cond_var.notify_all();
}

void resume() {
    std::unique_lock<std::mutex> lock(queue_mutex);
    paused = false;
    cond_var.notify_all();
}
```

Методи pause і resume, встановлює флаг paused відповідно до методу і викликає notify_all для повідомлення всіх потоків про зміну стану.

```
void stopQueue() {
    std::unique_lock<std::mutex> lock(queue_mutex);
    stop = true;
    tasks = {};
    cond_var.notify_all();
}
```

Зупиняє чергу, видаляє всі завдання з черги та сповіщає всіх потоків, що очікують на завдання.

```
class Task {
public:
    std::function<void()> func;
    int id;

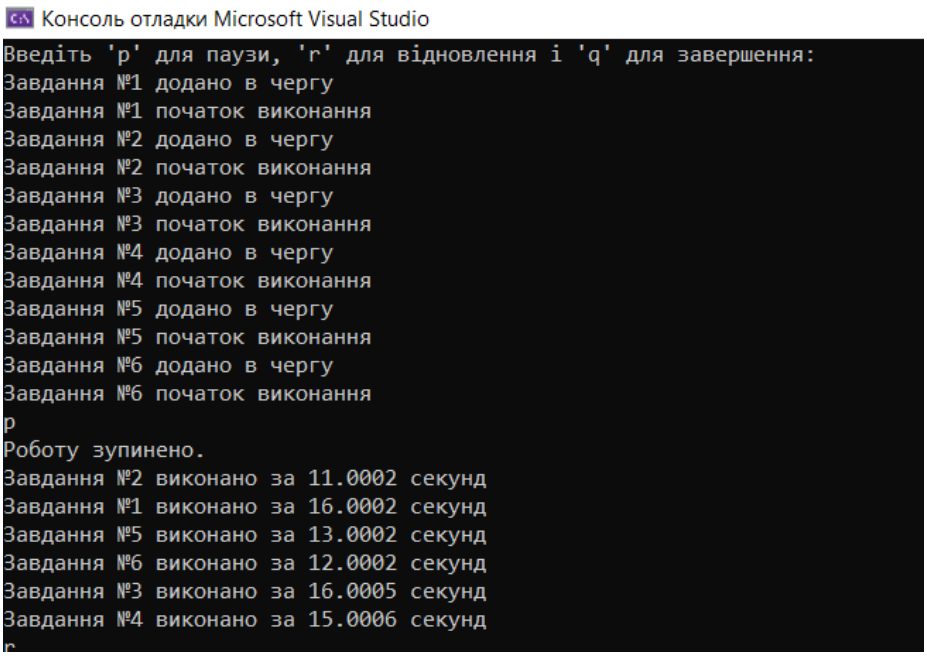
    Task(const std::function<void()>& f, int i)
    {
        func = f;
        id = i;
    }
};
```

Також створено клас таск (завдання має свою функцію та айді)

Тобто, логіка коду наступна:

Є клас Task, цей клас визначає структуру завдання, кожне завдання має функцію, яку потрібно виконати std::function<void()> func та ідентифікатор int id, що дозволяє відслідковувати і розрізняти завдання.
Клас TaskQueue, цей клас відповідає за управління чергою завдань, має методи addTask — додає нове завдання до черги (використовує std::unique_lock для синхронізації доступу до черги та condition_variable для повідомлення потоків про наявність нових завдань) getTask — отримує завдання з черги, Якщо черга порожня, потік буде чекати, доки не з'являться нові завдання або не буде активовано паузу/зупинку (pause і resume — керують станом паузи черги)
stopQueue — зупиняє чергу, забороняючи додавання нових завдань і очищаючи існуючі.
Клас ThreadPool який керує пулом робочих потоків. При ініціалізації створює фіксовану кількість потоків (за варіантом 8), кожен з яких виконує завдання з черги. worker — основна функція кожного потоку, що виконує завдання до зупинки системи, stop — зупиняє роботу всіх потоків, очікуючи на їх завершення. printStatistics — виводить статистику виконаних завдань.
Для додавання задач до потоків є окрема функція taskProducer, ця функція працює в окремому потоці і є виробником завдань, вона періодично додає завдання до черги, кожне з яких вимагає деякий час на виконання (симулюється за допомогою sleep). Враховує стан паузи черги, запобігаючи додаванню нових завдань під час паузи.

Результат виконання :

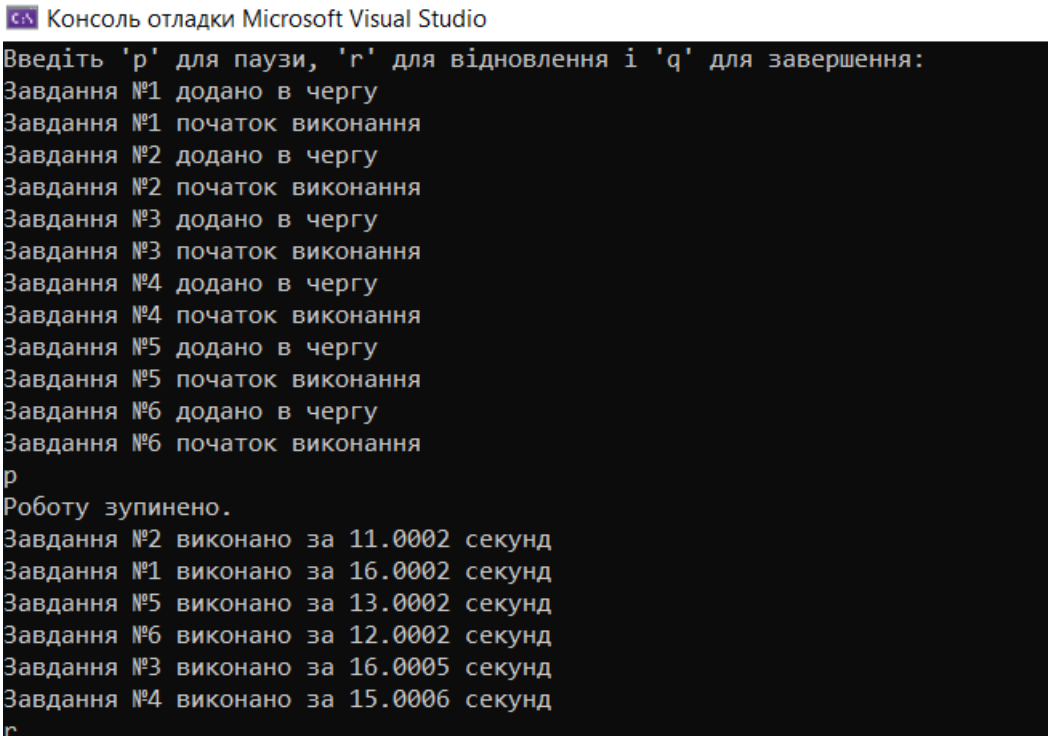


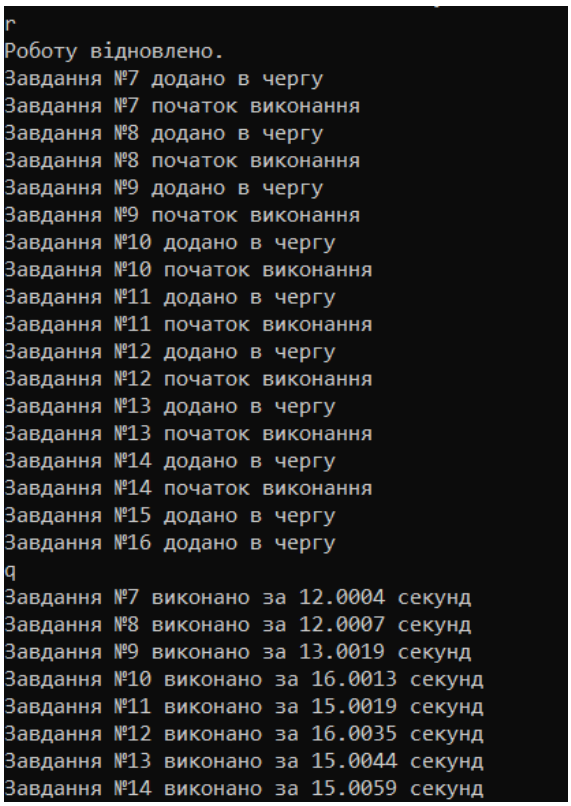
Бачимо, у нас є завдання, що безперервно додаються у чергу, виконавчи потоки беруть задачі з чергу та починають їх виконувати. У користувача є 3 опції керування, це p-пауза, r – відновлення, q – вихід.

Пауза pause()
Коли викликаєтьс метод pause(), флаг paused у TaskQueue встановлюється у true. Це призупиняє додавання нових задач у чергу з виробничого потоку (taskProducer). Виконавчі потоки (те, що у ThreadPool), які вже взяли задачу з черги, продовжать виконувати свої задачі до кінця. Якщо вони спробують взяти наступну задачу під час паузи, вони застрягнуть у чеканні на методі getTask(), який використовує умовну змінну cond_var для блокування потоку, доки черга не буде відновлена або зупинена.

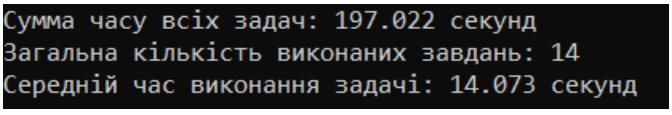
Відновлення resume()
Виклик resume() знімає паузу, встановлюючи paused у false. Це сповіщає всі потоки, які чекають на завдання (через cond_var.notify_all()), і вони можуть продовжити обробку задач із черги.

Зупинка stopQueue()
Коли викликається stopQueue(), флаг stop встановлюється у true. Це миттєво призводить до припинення додавання нових задач у чергу та зупиняє виробничий потік. Черга очищається (tasks = {}), тож всі невиконані задачі втрачаються. Виконавчі потоки, які чекають на нові задачі або закінчують свої поточні задачі, будуть припинені після виконання поточних завдань. Після того як усі задачі завершені, вони також зупиняться.





Також, після завершення роботи, для виводу статистики на екран було зроблено функцію void printStatistics()



Яка відображає кількість виконаних завдань, середній час виконання задачі, та сумму часу всіх задач.

Зробимо додавання задач у чергу кожну секунду, та подивимося на результати

Кількість завдань	Завдань виконано	Середній час виконання
30	19	13.5
70	44	13.4
100	63	13.04

Висновок.

У ході виконання лабораторної роботи №2 ми набули практичних навичок роботи з базовими примітивами синхронізації, такими як мютекси і умовні змінні. Було розроблено клас TaskQueue для управління чергою завдань, де кожне завдання додавалося асинхронно, а також клас ThreadPool, який відповідає за управління пулом потоків. Використання пулу потоків дозволяє оптимізувати використання системних ресурсів, зменшуючи накладні витрати на створення та знищення потоків, адже потоки ініціалізуються лише один раз і перевикористовуються для обробки множини завдань.

Також було зроблено механізми контролю за станом виконання потоків, включаючи мозливість паузи і зупинки виконання завдань, що дозволяє динамічно управляти потоками відповідно до змін у завантаженості системи або зовнішніх умов. При зупинці системи потоки можуть або завершити виконання поточних завдань, або негайно припинити роботу, що забезпечує гнучкість у управлінні ресурсами.

В кінці був зроблений аналіз отриманих даних стосовно часу, та кількості виконаних завдань

Додаток 1

```
#include <iostream>
#include <vector>
#include <queue>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <functional>
#include <chrono>
#include <windows.h>

class Task {
public:
    std::function<void()> func;
    int id;

    Task(const std::function<void()>& f, int i)
    {
        func = f;
        id = i;
    }
};

class TaskQueue {
private:
    std::queue<Task> tasks;
    std::mutex queue_mutex;
    std::condition_variable cond_var;
    bool stop = false;
    bool paused = false;
    int task_id = 0;

public:
    void addTask(const std::function<void()>& task) {
        std::unique_lock<std::mutex> lock(queue_mutex);
        if (!stop) {
```

```
            tasks.emplace(task, ++task_id);
            std::cout << "Завдання №" << task_id << " додано в чергу" <<
std::endl;
            cond_var.notify_one();
        }
    }

    Task getTask() {
        std::unique_lock<std::mutex> lock(queue_mutex);
        cond_var.wait(lock, [this]() { return stop || !tasks.empty() ||
paused; });
        if ((stop && tasks.empty()) || paused) {
            return Task(nullptr, 0);
        }
        Task task = tasks.front();
        tasks.pop();
        return task;
    }

    void pause() {
        std::unique_lock<std::mutex> lock(queue_mutex);
        paused = true;
        cond_var.notify_all();
    }

    void resume() {
        std::unique_lock<std::mutex> lock(queue_mutex);
        paused = false;
        cond_var.notify_all();
    }

    void stopQueue() {
        std::unique_lock<std::mutex> lock(queue_mutex);
        stop = true;
        tasks = {};
        cond_var.notify_all();
    }

    bool isStopped() {
        return stop;
    }

    bool isPaused() {
        return paused;
    }

};

class ThreadPool {
private:
    std::vector<std::thread> workers;
    TaskQueue& queue;
    std::vector<double> total_execution_time;
    std::vector<int> execution_count;
public:
    ThreadPool(TaskQueue& q, int workers_count) : queue(q) {
        total_execution_time.resize(workers_count, 0.0);
        execution_count.resize(workers_count, 0);

        for (int i = 0; i < workers_count; i++) {
            workers.emplace_back([this, i]() { this->worker(i); });
        }
    }

    void worker(int id) {
```

```
        while (!queue.isStopped()) {
            Task task = queue.getTask();
            if (task.func) {
                std::cout << "Завдання №" << task.id << " початок виконання" <<
std::endl;
                auto start_time = std::chrono::high_resolution_clock::now();
                task.func();
                auto end_time = std::chrono::high_resolution_clock::now();
                std::chrono::duration<double> elapsed = end_time - start_time;
                total_execution_time[id] += elapsed.count();
                execution_count[id]++;
                std::cout << "Завдання №" << task.id << " виконано за " <<
elapsed.count() << " секунд" << std::endl;
            }
        }
    }

    void stop() {
        queue.stopQueue();
        for (auto& worker : workers) {
            if (worker.joinable()) {
                worker.join();
            }
        }
    }

    void printStatistics() {
        double total_time = 0;
        int total_tasks = 0;

        for (int i = 0; i < total_execution_time.size(); i++) {
            total_time += total_execution_time[i];
            total_tasks += execution_count[i];
        }
        std::cout << "Сумма часу всіх задач: " << total_time << " секунд" <<
std::endl;
        std::cout << "Загальна кількість виконаних завдань: " << total_tasks <<
std::endl;
        if (total_tasks > 0) {
            std::cout << "Середній час виконання задачі: " << (total_time /
total_tasks) << " секунд" << std::endl;
        }
    }
};

void taskProducer(TaskQueue& queue) {
    while (!queue.isStopped()) {
        int sleep_time = 10 + rand() % 7;
        if (!queue.isPaused()) {
            queue.addTask([sleep_time]
{ std::this_thread::sleep_for(std::chrono::seconds(sleep_time)); });
        }
        std::this_thread::sleep_for(std::chrono::milliseconds(1000));
    }
}

int main() {
    srand(time(NULL));

    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);

    TaskQueue queue;
    ThreadPool pool(queue, 8);
```

```
    std::thread producer([&queue]() {taskProducer(queue); });

    char command;
    std::cout << "Введіть 'p' для паузи, 'r' для відновлення і 'q' для
завершення:" << std::endl;
    while (std::cin >> command && command != 'q') {
        if (command == 'p') {
            queue.pause();
            std::cout << "Роботу зупинено." << std::endl;
        }
        else if (command == 'r') {
            queue.resume();
            std::cout << "Роботу відновлено." << std::endl;
        }
    }

    pool.stop();
    pool.printStatistics();

    producer.join();
    return 0;
}
```